

Week Eight Technical Report

React Hooks, Routing, and Styling with Tailwind CSS

Trainee: David Karanja

Trainer: Brian Kemboi

Date: November 7, 2025

Teach2Give QA & QE Training Program

Overview

Week 8 continued our journey into frontend development, focusing on advanced React concepts. The main topics covered were React Hooks (`useState` and `useEffect`), React Router for page navigation, and Tailwind CSS for modern and responsive styling. These sessions built on the foundations established in the previous week and marked a shift towards creating dynamic, interactive, and visually appealing user interfaces.

Training Modules Covered

We began by exploring React Hooks, which allow the use of state and lifecycle features in functional components. The `useState` hook was used to manage component-level data, while `useEffect` helped us handle side effects such as fetching data, setting timers, and responding to state changes.

Later, we were introduced to React Router, a powerful library for managing navigation within single-page applications. We learned how to define routes using `BrowserRouter`, `Routes`, and `Route` components, and how to create seamless transitions between pages using `Link` and `NavLink`.

Finally, we learned Tailwind CSS, a utility-first CSS framework that allows fast UI design using predefined classes. Through Tailwind, we explored responsive layouts, color utilities, spacing, typography, and flexbox properties — all integrated directly into our React components.

Practical Exercise and Project

To apply these concepts, we enhanced the `ToDoApp` by implementing state management using `useState` and handled side effects with `useEffect`. We also practised navigation by adding multiple pages and routing between them using React Router. Tailwind CSS was applied to style components efficiently, ensuring responsiveness and clean layouts.

In our group project — the Bug Tracking System — we are still implementing several new features. A notable one is automatically assigning the creator of a project as the project lead, as well as including all team members assigned to a group along with their

roles and tasks. This required enforcing access control, ensuring that only the admin and project lead could update project details, or add and remove members.

Challenges Faced

The week's main challenge was mastering the flow of state and side effects within React components, particularly understanding how the `useEffect` dependencies influence renders. At the same time, we faced technical hurdles in integrating authorization and role-based access control within the Bug Tracking System. Implementing these features required aligning backend logic with user roles and permissions to ensure only project leads and admins could modify project details or manage members. Despite the initial complexity, we successfully managed to integrate these functionalities through teamwork and consistent testing. Over the weekend, we plan to incorporate the React concepts covered into the Bug Tracking System to begin building its frontend interface.

Recommendations

I recommend practicing on Hooks thoroughly, especially by building small projects that rely on dynamic data and side effects. Experimenting with React Router's documentation helps deepen understanding of route handling. For Tailwind CSS, using its official documentation and interactive examples is an excellent way to learn responsive design patterns efficiently. For backend-related projects, it is also helpful to plan role-based access logic early to avoid integration issues later.

Conclusion

This week solidified my understanding of React's core features and introduced me to essential tools for modern web development — Hooks, React Router, and Tailwind CSS. These skills are instrumental as we continue building the frontend for the Bug Tracking System project, ensuring it is well-structured, responsive, and user-friendly.

Signature:

David Karanja